

ENTERPRISE KUBERNETES MASTERY

AI Platform Engineering Handbook

PART III CONTAINERS

Docker, OCI, Runtimes, Image Security, and Supply Chain

Volume 3 of 16 Foundation Series

Prerequisites: Parts I and II

Edition 2025-2026

TABLE OF CONTENTS

- 1. Container Technology Overview 3
- 2. Docker Architecture Deep Dive 5
- 3. OCI Image and Runtime Specifications 9
- 4. Container Runtimes: containerd, CRI-O, gVisor, Kata . 14
- 5. Image Building: Multi-Stage, Distroless, Scratch .. 19
- 6. Container Registry Architecture 24
- 7. Image Optimisation Strategies 27
- 8. Secure Software Supply Chain 30
- 9. Image Signing: Cosign and Notary v2 33
- 10. SBOM Generation and Management 37
- 11. SLSA Framework for Container Builds 40
- 12. Vulnerability Scanning in Depth 43
- 13. Runtime Container Security 47
- 14. Container Anti-Patterns and Remediation 52
- 15. Hands-On Exercises 56

CHAPTER 1

Container Technology Overview

This part provides a comprehensive technical treatment of container technologies as deployed in production Kubernetes environments. The focus is depth: not just what containers are, but how every layer of the container stack works internally and how it interacts with Kubernetes.

Modern Kubernetes clusters use containerd or CRI-O as their runtime, interact with OCI-compliant registries, enforce supply chain security through image signing and SBOM, and apply multiple layers of runtime security through seccomp, AppArmor, and eBPF. The naive assumption that Kubernetes uses Docker is false and outdated.

Container Stack Architecture

```
Complete container technology stack (top to bottom): DEVELOPER TOOLING Dockerfile /  
Containerfile / BuildKit / ko / Jib / Buildpacks | IMAGE DISTRIBUTION OCI Image Format ->  
Registry (Harbor, ECR, GAR, GCR, Quay) Cosign signing -> Notary v2 -> SBOM attestation ->  
Rekor log | KUBERNETES ORCHESTRATION API Server -> Scheduler -> kubelet | CRI INTERFACE (gRPC)  
containerd (default) / CRI-O | OCI RUNTIME INTERFACE runc (default) / crun / kata-runtime /  
runsc (gVisor) | LINUX KERNEL Namespaces + cgroups + OverlayFS + seccomp + capabilities + eBPF
```

Key Insight

Each layer of this stack is independently swappable via open standards. An enterprise can use BuildKit for builds, Harbor for registry, containerd as CRI, and Kata Containers as OCI runtime -- all standards-compliant, all interoperable, no vendor lock-in at any layer. Understanding each layer enables informed architectural decisions and precise troubleshooting.

CHAPTER 2

Docker Architecture Deep Dive

Docker is both a company and a set of tools. Over time, the original monolithic Docker daemon has been decomposed through standardisation. Understanding this decomposition explains why Kubernetes no longer depends on Docker as a runtime and clarifies how each component of the modern container stack relates to Docker's original architecture.

Docker Component Architecture (Before and After Decomposition)

```
ORIGINAL (pre-2016 monolith): docker CLI --> dockerd (monolithic daemon) handles: images, containers, volumes, networks, build
CURRENT (decomposed): docker CLI | REST API / Unix socket
dockerd (thin orchestration layer) | gRPC containerd <-- donated to CNCF 2017; manages container lifecycle | exec shim containerd-shim-runc-v2 <-- per-container daemonized shim | OCI bundle runc <-- donated to OCI 2015; reference runtime implementation | Linux syscalls Linux Kernel (namespaces, cgroups, OverlayFS)
KUBERNETES PATH (no Docker at all): kubelet | CRI gRPC containerd (or CRI-O) | exec shim runc / kata / runsc | Linux Kernel
```

The Dockershim Removal -- Kubernetes 1.24 (May 2022)

Kubernetes originally included a Docker-specific shim (dockershim) that translated CRI calls to Docker API calls. This was deprecated in 1.20 (December 2020) and removed in 1.24. Impact assessment:

- * **Zero impact on most workloads:** OCI-compliant images built with Docker run identically under containerd or CRI-O. No image format changes required.
- * **Node tooling migration:** Tooling accessing /var/run/docker.sock on nodes (Docker-in-Docker CI, some monitoring agents) required migration to /run/containerd/containerd.sock or crictl commands.
- * **Managed cluster transparency:** GKE, EKS, AKS migrated their node images to containerd automatically; most managed cluster users saw no impact.
- * **Self-managed cluster action required:** Clusters running dockershim needed to update node images or reconfigure runtime before upgrading to 1.24.

BuildKit -- The Modern Docker Build Engine

BuildKit (default in Docker 23+, Docker Desktop 4.0+) is a massively improved build engine providing parallel stage execution, efficient cache management, and secure secret handling during builds:

Feature	Description	Kubernetes Impact
Parallel stages	Independent FROM stages build simultaneously	Faster CI pipelines, lower build time
Registry cache backend	Cache layers in OCI registry, not local disk	Consistent cache across CI runners
Secret mounts	RUN --mount=type=secret; secret not in layer	API keys never persist in image layers

Feature	Description	Kubernetes Impact
SSH agent mounts	RUN --mount=type=ssh; private repos without keys in image	Secure private dependency access
Rootless builds	Build without root; runs as unprivileged user	Secure CI runners, Tekton pipelines
Multi-platform	--platform linux/amd64,linux/arm64 in single build	Single CI step for arm64 K8s nodes
Inline cache	Embed cache metadata in image manifest	Registry-based cache sharing
Reproducible builds	Deterministic layer hashes with --no-cache-filter	SLSA provenance verification

Production Dockerfile with BuildKit Features

```
# syntax=docker/dockerfile:1.6 # Stage 1: Dependency resolution (cached separately) FROM
golang:1.22-alpine AS deps WORKDIR /build COPY go.mod go.sum ./ # Cache Go module download
across builds RUN --mount=type=cache,target=/root/go/pkg/mod \ go mod download # Stage 2:
Build (parallel with deps if independent) FROM deps AS builder COPY . . # Cache Go build
cache; use SSH for private modules RUN --mount=type=cache,target=/root/go/pkg/mod \
--mount=type=cache,target=/root/.cache/go-build \ --mount=type=ssh \ CGO_ENABLED=0 GOOS=linux
go build \ -trimpath -ldflags='-s -w' \ -o /app ./cmd/server # Stage 3: Minimal production
image FROM gcr.io/distroless/static-debian12:nonroot COPY --from=builder /app /app USER
nonroot:nonroot EXPOSE 8080 ENTRYPOINT ["/app"]
```

CHAPTER 3

OCI Image and Runtime Specifications

OCI Image Specification -- Internal Structure

The OCI Image Specification defines how container images are stored and distributed. An image consists of three content-addressed components stored in a registry:

```
1. IMAGE INDEX (optional, for multi-architecture images): Manifest list pointing to
per-architecture manifests. mediaType: application/vnd.oci.image.index.v1+json
2. IMAGE MANIFEST (per-architecture): mediaType: application/vnd.oci.image.manifest.v1+json {
  schemaVersion: 2, config: { digest: sha256:CONFIG_HASH, mediaType: ...config.v1+json },
  layers: [ { digest: sha256:L1, mediaType: ...layer.v1.tar+gzip, size: 12345 }, { digest:
sha256:L2, mediaType: ...layer.v1.tar+gzip, size: 45678 } ] }
3. IMAGE CONFIGURATION: mediaType: application/vnd.oci.image.config.v1+json { architecture: amd64, os: linux, config:
{ Cmd: ["/app"], Env: ["PATH=/usr/local/bin"], User: 1000:1000, WorkingDir: /app,
ExposedPorts: {8080/tcp: {} } }, rootfs: { type: layers, diff_ids: [sha256:L1_UNCOMPRESSED,
sha256:L2_UNCOMPRESSED] }, history: [ { created_by: RUN apt-get install nginx -y }, ... ] }
```

Content Addressing -- Why Every Digest Matters

Every blob in an OCI registry is identified by the SHA-256 digest of its content. This content addressing provides fundamental security properties:

- * **Immutability:** A blob with a given sha256 digest always contains identical bytes. Content cannot change without the digest changing -- detectable tampering.
- * **Global deduplication:** The nginx:alpine base layer shared by 1000 images is stored once per registry and once per node. Storage scales with unique layers, not with image count.
- * **Verifiable pulls:** Pulling by digest (image@sha256:...) guarantees the exact content you expect. Pulling by tag does not -- tags are mutable pointers.
- * **Build cache precision:** Build systems use layer digests to determine cache validity. If the digest matches a cached layer, the build step is skipped exactly.

Critical: Always Pull by Digest in Production

In production Kubernetes deployments, always reference images by digest rather than tag. Tags like :latest or :v1.2.3 are mutable -- the underlying image can change without the tag changing, enabling supply chain attacks. Use: image: myregistry.io/myapp@sha256:abc123... in production PodSpecs. Kyverno can enforce this policy cluster-wide.

OCI Distribution Specification -- Registry API

```
The OCI Distribution Spec defines the HTTP API for pushing/pulling images:
# PULL WORKFLOW:
GET /v2/ # Registry discovery
GET /v2/myapp/manifests/v1.0 # Resolve tag to manifest
GET /v2/myapp/manifests/sha256:HASH # Fetch manifest by digest
GET /v2/myapp/blobs/sha256:L1 # Download layer blob
GET /v2/myapp/blobs/sha256:CONFIG # Download image config
# PUSH WORKFLOW:
POST /v2/myapp/blobs/uploads/ # Initiate layer upload
PUT /v2/myapp/blobs/uploads/UUID?digest=sha256:L1 # Complete upload
PUT /v2/myapp/manifests/v1.0 # Create/update tag
# REFERRERS (signatures, SBOMs, attestations): GET
```

```
/v2/myapp/referrers/sha256:MANIFEST # List artifacts referencing image # TAG IMMUTABILITY  
(Harbor, ECR, GAR support): # Configure registry to reject overwrites of existing tags #  
Forces use of new tags for new content -- supply chain hygiene
```

CHAPTER 4

Container Runtimes: containerd, CRI-O, gVisor, Kata

containerd -- The Production Default

containerd (CNCF graduated) is the container runtime used by default in GKE, EKS, AKS, and most modern Kubernetes distributions. It manages the complete container lifecycle: image pulling, storage (snapshots via OverlayFS), container execution, networking handoff to CNI, and metrics.

```
containerd internal architecture: containerd daemon (/run/containerd/containerd.sock) | +--
CRI Plugin (gRPC server implementing Kubernetes CRI API) | Translates kubelet CRI calls to
containerd native API | +-- Content Store (immutable, content-addressed blob storage) |
/var/lib/containerd/io.containerd.content.v1.content/blobs/sha256/ | +-- Snapshot Service
(OverlayFS layer management) | /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/ |
+-- Image Service (image metadata, pull orchestration) | +-- Task Service (running containers)
| +-- containerd-shim-runc-v2 (per-container daemon, survives containerd restart) | +-- runc
create -> runc start -> runc delete | +-- Events Service (lifecycle events streamed to
kubelet) | +-- GC (garbage collection of unused snapshots and content)
```

Key containerd Production Configuration

```
# /etc/containerd/config.toml -- critical production settings version = 2
[plugins.'io.containerd.grpc.v1.cri'] sandbox_image = 'registry.k8s.io/pause:3.9'
max_container_log_line_size = 16384 [plugins.'io.containerd.grpc.v1.cri'.containerd]
snapshotter = 'overlayfs' default_runtime_name = 'runc'
[plugins.'io.containerd.grpc.v1.cri'.containerd.runtimes.runc] runtime_type =
'io.containerd.runc.v2' [plugins.'io.containerd.grpc.v1.cri'.containerd.runtimes.runc.options]
SystemdCgroup = true # MUST match kubelet --cgroup-driver=systemd
[plugins.'io.containerd.grpc.v1.cri'.containerd.runtimes.kata] runtime_type =
'io.containerd.kata.v2' [plugins.'io.containerd.grpc.v1.cri'.registry.mirrors]
[plugins.'io.containerd.grpc.v1.cri'.registry.mirrors.'docker.io'] endpoint =
['https://harbor.internal.corp/v2/dockerhub-proxy'] [plugins.'io.containerd.gc.v1.scheduler']
deletion_threshold = 256 pause_threshold = 0.02 startup_delay = '100ms'
```

CRI-O -- Kubernetes-Native Runtime

CRI-O (Red Hat, Kubernetes-native) is a lightweight alternative to containerd that implements CRI but nothing else. Unlike containerd (which also serves Docker-compatible workloads), CRI-O is designed purely for Kubernetes. It is the default runtime in Red Hat OpenShift.

CRI-O vs containerd comparison:

Dimension	containerd	CRI-O
Primary use	General-purpose (Docker + K8s)	Kubernetes-only
Image pull	Built-in	Uses containers/image library
Storage	Built-in snapshotter	Uses containers/storage

Dimension	containerd	CRI-O
Networking	CNI plugin handoff	CNI plugin handoff (identical)
OCI runtime	Pluggable (runc, kata, runsc)	Pluggable (identical)
Docker compat	Yes (dockerd can use it)	No Docker support
Default in	GKE, EKS, AKS, k3s, RKE2	OpenShift 4.x
Config	/etc/containerd/config.toml	/etc/crio/crio.conf
Debug CLI	crictl, ctr, nerdctl	crictl, podman

gVisor -- Application Kernel Sandboxing

gVisor (open-sourced by Google 2018) implements a user-space kernel that intercepts system calls made by container processes, providing strong isolation without full VM overhead. GKE Autopilot uses gVisor for all workloads.

```
gVisor architecture and trade-offs: Container Process | system calls [gVisor Sentry] --
user-space kernel in Go | proxied file I/O [gVisor Gofer] -- filesystem access mediator |
minimal real syscalls Linux Host Kernel Two execution modes: ptrace: intercepts syscalls via
ptrace (compatible, ~20% perf overhead) KVM: hardware assist via /dev/kvm (faster, requires
nested virt support) Security properties: * Separate kernel per container (blast radius
containment) * ~95% Linux syscall coverage (some apps incompatible) * No shared kernel state
between containers Performance characteristics: * CPU-bound workloads: ~5% overhead *
Syscall-heavy (file I/O, network): ~30% overhead * Memory allocation: moderate overhead * Not
suitable for GPU workloads (no CUDA support)
```

Kata Containers -- VM-Level Isolation in Kubernetes

Kata Containers runs each Pod inside a lightweight virtual machine, providing hypervisor-level isolation while maintaining full OCI/CRI compatibility. The standard approach for multi-tenant Kubernetes where different tenants' workloads must not share a Linux kernel.

```
Kata Containers architecture: Kubernetes Pod spec (runtimeClassName: kata-qemu) | containerd /
CRI-O --> kata-runtime (OCI shim) | QEMU / Firecracker / Cloud Hypervisor (VMM) | MicroVM
(minimal Linux kernel inside VM) | kata-agent (process manager inside VM) | Container process
(PID 1 inside VM kernel) RuntimeClass configuration: apiVersion: node.k8s.io/v1 kind:
RuntimeClass metadata: { name: kata-qemu } handler: kata-qemu overhead: podFixed: { memory:
120Mi, cpu: 250m } # VM overhead scheduling: nodeClassification: tolerations: [{ key: kata,
operator: Exists }]
```

Runtime Selection Decision Matrix

Runtime	Isolation Boundary	Perf Overhead	Startup	GPU	Best Use Case
runc	Linux namespaces	less than 1%	50ms	Yes (CDI)	Trusted workloads, standard production

Runtime	Isolation Boundary	Perf Overhead	Startup	GPU	Best Use Case
crun (C impl)	Linux namespaces	less than 0.5%	30ms	Yes (CDI)	High-density, perf-critical
gVisor (runsc)	User-space kernel	5-30%	200ms	No	Untrusted/SaaS, multi-tenant
Kata + QEMU	Full VM (QEMU)	5-10%	500ms	GPU passthrough	Regulated, high-security multi-tenant
Kata + Firecracker	MicroVM	3-7%	125ms	Emerging	Serverless-style, fast cold start
Kata + Cloud-HV	Lightweight VM	4-8%	200ms	Yes (vfi o-pci)	Performance + security balance

CHAPTER 5

Image Building: Multi-Stage, Distroless, Scratch

Multi-Stage Builds -- Production Pattern

Multi-stage builds are the single most important Dockerfile pattern for production images. They separate build environments (compilers, build tools, test frameworks) from runtime environments (minimal, hardened). The result: smaller attack surface, faster pulls, lower CVE count.

Go Application -- Scratch Image

```
# syntax=docker/dockerfile:1.6 FROM golang:1.22-alpine AS builder WORKDIR /build COPY go.mod go.sum ./ RUN --mount=type=cache,target=/root/go/pkg/mod go mod download COPY . . RUN --mount=type=cache,target=/root/go/pkg/mod \ --mount=type=cache,target=/root/.cache/go-build \ CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build \ -trimpath -ldflags='-s -w' -o /app ./cmd/server FROM scratch COPY --from=builder /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/ COPY --from=builder /usr/share/zoneinfo /usr/share/zoneinfo COPY --from=builder /app /app USER 65534:65534 # nobody ENTRYPOINT ["/app"] # Result: ~8MB image, zero shell, zero OS tools, minimal CVE surface
```

Java Application -- Distroless with Custom JRE

```
# syntax=docker/dockerfile:1.6 FROM eclipse-temurin:21-jdk AS build WORKDIR /app COPY pom.xml . RUN --mount=type=cache,target=/root/.m2 mvn dependency:resolve -q COPY src ./src RUN --mount=type=cache,target=/root/.m2 mvn package -DskipTests -q # Build minimal JRE with only required modules FROM eclipse-temurin:21-jdk AS jre-builder COPY --from=build /app/target/app.jar /tmp/app.jar RUN jdeps --ignore-missing-deps -q --recursive \ --multi-release 21 --print-module-deps \ /tmp/app.jar > /tmp/modules.txt RUN jlink --add-modules $(cat /tmp/modules.txt) \ --strip-debug --compress 2 \ --no-header-files --no-man-pages \ --output /custom-jre FROM gcr.io/distroless/base-debian12:nonroot COPY --from=jre-builder /custom-jre /opt/java COPY --from=build /app/target/app.jar /app.jar USER nonroot:nonroot ENTRYPOINT ["/opt/java/bin/java", "-jar", "/app.jar"] # Result: ~95MB vs ~800MB JDK image; no shell, no apt, no curl
```

Distroless Images Reference

Image	Contents	Approx Size	Use For
gcr.io/distroless/static-debian12	CA certs, timezone data only	2 MB	Go (static), Rust
gcr.io/distroless/base-debian12	glibc, libssl, openssl, ca-certs	20 MB	Go (CGO), C/C++
gcr.io/distroless/cc-debian12	C++ runtime + base	20 MB	C++ applications
gcr.io/distroless/java21-debian12	JRE 21 + base	195 MB	Java 21
gcr.io/distroless/python3-debian12	Python 3 + base	55 MB	Python 3.x

Image	Contents	Approx Size	Use For
gcr.io/distroless/nodejs20-debian12	Node.js 20 + base	120 MB	Node.js 20
:nonroot variants	Same + runs as UID 65532 by default	Same	Security-first
:debug variants	Same + busybox shell (DO NOT USE IN PROD)	Varies	Debugging only

Alternative Build Tools for Kubernetes

- * **ko (Google)**: Builds Go binaries directly into OCI images without a Dockerfile. Zero-config, produces distroless images by default, integrates with Cosign for signing.
- * **Jib (Google)**: Builds Java images without Docker daemon. Produces layered images separating dependencies, resources, and classes for optimal caching. Maven/Gradle plugins.
- * **Cloud Native Buildpacks**: Detects language, applies runtime best practices, patches OS-level CVEs without Dockerfile changes. Heroku buildpacks, Paketo, Google Buildpacks.
- * **Bazel**: Hermetic, reproducible builds at Google scale. Produces identical binary artifacts regardless of build environment -- essential for SLSA L3+ supply chain.
- * **Nix / nixpkgs**: Purely functional, reproducible package management. Enables 100% reproducible container images with cryptographic build provenance.

CHAPTER 6

Container Registry Architecture

Enterprise Registry Architecture

A container registry stores and serves OCI images and artifacts (Helm charts, SBOMs, signature attestations). Enterprise architectures require careful registry design for security, performance, compliance, and business continuity.

Registry	Type	Key Enterprise Features	Best For
Harbor	Self-hosted OSS (CNCF)	Trivy scanning, RBAC, proxy cache, replication, webhooks, OIDC	On-prem / air-gap / full control
AWS ECR	Managed cloud	IAM auth, lifecycle policies, cross-region, VPC endpoints, Inspector scanning	AWS-native K8s
GCP Artifact Registry	Managed cloud	Workload Identity, VPC-SC, CMEK, multi-format (Docker/Helm/npm/Maven)	GKE workloads
Azure Container Registry	Managed cloud	Geo-replication, Private Link, Tasks CI, Defender scanning, RBAC	AKS workloads
Quay.io / Project Quay	Cloud + self-hosted	Clair scanning, robot accounts, org teams, build triggers, mirrors	OpenShift / hybrid
GitHub Container Registry	Cloud (ghcr.io)	GitHub Actions OIDC, fine-grained PAT, public+private, packages	OSS / GitHub-native
Zot	Self-hosted OSS	OCI-native only, ultra-lightweight, S3 backend, pluggable	Edge / air-gap / minimal

Enterprise Registry Reference Architecture

```
Recommended multi-tier architecture for regulated enterprises: TIER 1: Source registries (public internet) docker.io / gcr.io / ghcr.io / quay.io / registry.k8s.io | [Egress firewall: explicit allow-list of approved source registries] | TIER 2: Internal Harbor (on-prem or VPC) +-- Proxy cache projects (pull-through with auto-scanning) | Scanned on first pull; results cached 24h | Blocked if CRITICAL CVEs found (configurable policy) +-- Internal CI/CD projects (built + scanned images) | Cosign signatures attached; SBOMs attached +-- Promotion pipeline dev -> staging -> production (digest-pinned at each stage) | [OPA/Kyverno: only allow images from harbor.internal.corp] | TIER 3: Kubernetes Clusters containerd mirror config -> harbor.internal.corp Admission webhook -> verify Cosign signature before scheduling
```

Image Lifecycle Policies

Unmanaged registries accumulate stale images rapidly. Implement retention policies:

- * **Tag retention:** Keep last N tags per repository (e.g., last 10 production tags, last 3 per branch in dev)

- * **Untagged image cleanup:** Delete untagged images after 7 days -- they are build intermediates with no legitimate use
- * **Age-based deletion:** Delete images older than 90 days in dev/staging repositories automatically
- * **Usage-based retention:** Harbor and ECR can track which images were actually pulled; retain pulled images longer than unpulled
- * **Immutable production tags:** Configure registries to reject tag overwrites for production tags (v1.2.3 must always point to same digest)

CHAPTER 7

Image Optimisation Strategies

Layer Cache Optimisation

Dockerfile layer ordering is one of the highest-impact optimisation decisions. Layers that change frequently must be placed after layers that change rarely to maximise cache hit rates in CI systems.

```
ANTI-PATTERN (cache invalidated on every commit): COPY . . # changes every commit RUN pip
install -r requirements.txt # rebuilds every commit! OPTIMAL (dependencies cached separately):
COPY requirements.txt . # changes only with dep updates RUN pip install -r requirements.txt #
cached until deps change COPY . . # only this layer rebuilds per commit LAYER ORDERING BY
CHANGE FREQUENCY (least -> most frequent): FROM base # changes: major upgrades (quarterly) RUN
install-os-packages # changes: security patches (monthly) COPY dep-manifests . # changes: dep
updates (weekly) RUN install-deps # changes: dep updates (weekly) COPY source-code . #
changes: every commit RUN build-app # changes: every commit
```

Image Size Reduction Techniques

Technique	Impact	Implementation
Multi-stage builds	60-95% size reduction	Separate build/runtime stages; copy only artifacts
Distroless base images	50-80% vs full OS	Use gcr.io/distroless/* instead of ubuntu/debian
Scratch base	Maximum reduction	CGO_ENABLED=0 Go binary + minimal files only
Combine RUN commands	Avoid intermediate layers	RUN apt update && apt install -y X && rm -rf /var/lib/apt/lists/*
strip debug symbols	20-50% binary size	Go: -ldflags='-s -w'; C/C++: strip binary
--no-install-recommends	10-30% package size	apt-get install --no-install-recommends
.dockerignore	Faster builds	Exclude .git, tests, docs, node_modules from context
dive inspection	Find hidden large layers	dive myimage:tag shows layer contents and waste

Kubernetes-Specific Image Considerations

- * **Startup speed:** Kubernetes probes (startupProbe, readinessProbe) fire before the application is ready. Smaller images pull faster on cold starts (new node scale-out), reducing time to ready. Use image pull policies correctly: IfNotPresent for production (avoids re-pull on restart), Always for latest.

- * **Multi-arch images:** Build for both linux/amd64 and linux/arm64 to support mixed node pools. AWS Graviton, Azure Ampere, and GKE Tau nodes use arm64 and cost 20-40% less than x86 equivalents.
- * **Non-root requirement:** Pod Security Standards Restricted profile requires runAsNonRoot: true. Build images with non-root user: USER 1000:1000 and ensure application can bind to ports above 1024.
- * **Read-only filesystem:** Enable readOnlyRootFilesystem: true and use emptyDir or PVCs for write paths. Image layers are read-only by design; the writable layer is an ephemeral overlay.
- * **Ephemeral storage limits:** Container writable layer + emptyDir counts against ephemeral storage limit. Avoid writing large files to container filesystem; use PVCs for persistent data.

CHAPTER 8

Secure Software Supply Chain

The Supply Chain Threat Model

SolarWinds (2020), Codecov (2021), XZ Utils (2024), and repeated npm/PyPI package compromises demonstrated that the software supply chain is now the primary enterprise attack vector. Container images are especially vulnerable because they bundle hundreds of dependencies -- any one of which could be compromised without obvious indication.

Attack Vector	Example	Container Impact	Defence
Compromised base image	malicious nginx:alpine pushed to Docker Hub	All images FROM that base inherit malware	Pin by digest; use internal mirrors only
Dependency confusion	Attacker publishes malicious 'mycompany-utils' on PyPI	Build installs attacker package over internal one	Private registries; exact version pins; hash verification
Build system compromise	CI runner compromised; backdoor injected during build	Malicious binary in legitimate-looking image	Ephemeral builders; signed provenance; SLSA L3
Registry hijack	Leaked registry credentials; attacker overwrites production tag	Pods run attacker image next restart	Immutable tags; Cosign signatures; RBAC on registry
Typosquatting	nginx (no namespace) vs library/nginx on Docker Hub	Developer accidentally pulls compromised image	Internal mirror as only allowed source; OPA policy
Transit attack	MITM on HTTP registry pull	Different image delivered than expected	TLS everywhere; pull by digest; signature verification

Supply Chain Security -- Defence in Depth

Layer	Controls	Primary Tools
Source	Signed commits, branch protection, 2-person review	Git GPG signing, GitHub/GitLab branch rules
Dependencies	Lock files, hash pinning, private mirrors, scanning	pip-tools, poetry.lock, npm shrinkwrap, Renovate

Layer	Controls	Primary Tools
Build	Ephemeral runners, hermetic builds, signed provenance	SLSA builder, Tekton Chains, BuildKit
Image	Vuln scan, Cosign sign, SBOM generate, tag immutability	Trivy, Grype, Cosign, Syft, Harbor
Registry	RBAC, audit log, replication, proxy cache with scanning	Harbor, ECR, GAR with scanning policies
Deployment	Signature verification, policy-as-code, digest pinning	Kyverno, OPA Gatekeeper, Sigstore
Runtime	Syscall filter, behaviour monitoring, anomaly detection	Falco, Tetragon, seccomp, AppArmor

CHAPTER 9

Image Signing: Cosign and Notary v2

Sigstore -- The Signing Ecosystem

Sigstore is an OpenSSF project providing free, open infrastructure for signing software artifacts. It consists of three services that together enable keyless, identity-based signing tied to OIDC identity providers (GitHub, Google, Microsoft):

- * **Cosign:** CLI and Go library for signing and verifying OCI images and artifacts. Signs image manifests; stores signatures as OCI referencers (co-located in registry).
- * **Fulcio:** Certificate Authority that issues short-lived code signing certificates tied to OIDC identities. A GitHub Actions job gets a cert for its GitHub Actions identity. No long-lived key management required.
- * **Rekor:** Append-only, tamper-evident transparency log. Every signing event is recorded. Enables anyone to audit who signed what and when. analogous to Certificate Transparency for TLS certificates.

Keyless Signing Workflow in CI/CD

```
Keyless signing workflow (GitHub Actions): 1. GitHub Actions job starts; GitHub provides OIDC token (identity: https://github.com/myorg/myrepo/.github/workflows/build.yml) 2. cosign sign command: a. Generates ephemeral key pair (in memory; never persisted) b. Requests certificate from Fulcio CA c. Fulcio verifies OIDC token; issues cert binding identity to ephemeral pubkey d. cosign signs image manifest digest with ephemeral private key e. Uploads (signature + Fulcio cert) to registry as OCI artifact f. Records entry in Rekor transparency log g. Ephemeral private key discarded 3. Verification at admission time (Kyverno/policy controller): a. Fetch image manifest digest b. Retrieve signature from registry (OCI referrer) c. Verify signature against Fulcio cert d. Check cert identity matches expected GitHub org/repo e. Verify Rekor log entry exists (non-repudiable audit trail) f. ACCEPT or REJECT pod based on verification result
```

Cosign Integration -- GitHub Actions

```
# .github/workflows/release.yml name: Build and Sign on: [push] jobs: build-sign: runs-on: ubuntu-latest permissions: contents: read id-token: write # Required for OIDC/keyless signing packages: write steps: - uses: sigstore/cosign-installer@v3 - name: Build and push image id: build uses: docker/build-push-action@v5 with: push: true tags: ghcr.io/myorg/myapp:latest # Always capture digest for signing by digest (not tag) - name: Sign image by digest run: | cosign sign --yes \ ghcr.io/myorg/myapp@${{ steps.build.outputs.digest }}
```

Enforcing Signatures with Kyverno

```
apiVersion: kyverno.io/v1 kind: ClusterPolicy metadata: name: require-signed-images spec: validationFailureAction: Enforce background: false rules: - name: verify-cosign-signature match: any: - resources: kinds: [Pod] namespaces: [production, staging] verifyImages: - imageReferences: - ghcr.io/myorg/* - harbor.internal.corp/* attestors: - entries: - keyless: subject: https://github.com/myorg/* issuer: https://token.actions.githubusercontent.com rekor: url: https://rekor.sigstore.dev # Also pin to digest automatically: mutatedDigest: true verifyDigest: true
```

CHAPTER 10

SBOM Generation and Management

Why SBOMs Are Now Mandatory

A Software Bill of Materials (SBOM) is a machine-readable inventory of all software components in an application or container image: package names, versions, checksums, licenses, and dependency relationships. Regulatory drivers have made SBOMs mandatory in many contexts:

- * **US EO 14028 (2021):** Executive Order on Improving Cybersecurity requires SBOMs for software sold to US federal government agencies.
- * **EU Cyber Resilience Act (2024):** Requires SBOMs for digital products sold in the EU market; includes container-based software products.
- * **NIST SP 800-218:** Secure Software Development Framework includes SBOM generation as a recommended practice for secure software supply chains.
- * **Financial services regulators:** OCC, FFIEC, and PRA guidance increasingly references SBOMs as part of third-party software risk management.

SBOM Formats

Format	Maintained By	Output Formats	Ecosystem Adoption
SPDX 2.3	Linux Foundation / ISO 5962	JSON, RDF, YAML, tag-value	GitHub, NTIA, many scanners
CycloneDX 1.6	OWASP	JSON, XML, Protobuf	Syft, Gype, Trivy, OWASP
SWID	ISO/IEC 19770-2	XML	Enterprise software assets, less common

Generating and Attaching SBOMs

```
# Generate SBOM with Syft (supports SPDX and CycloneDX): syft scan
harbor.internal.corp/myapp:v1.2.3 \ -o cyclonedx-json=sbom.cdx.json \ -o
spdx-json=sbom.spdx.json # Attach SBOM as OCI attestation (Cosign): cosign attest \ --yes \
--predicate sbom.cdx.json \ --type cyclonedx \ harbor.internal.corp/myapp@sha256:DIGEST #
Verify and retrieve SBOM attestation: cosign verify-attestation \ --type cyclonedx \
--certificate-identity-regexp 'github.com/myorg' \ --certificate-oidc-issuer
'https://token.actions.githubusercontent.com' \ harbor.internal.corp/myapp@sha256:DIGEST \ |
jq -r '.payload' | base64 -d | jq '.predicate' # Scan SBOM for known CVEs (without re-scanning
image): gype sbom:sbom.cdx.json --fail-on critical
```

SBOM in Kubernetes Admission Control

```
SBOMs enable policy decisions at admission time: Example Kyverno policy to require SBOM
attestation: apiVersion: kyverno.io/v1 kind: ClusterPolicy metadata: name:
require-sbom-attestation spec: validationFailureAction: Enforce rules: - name:
check-sbom-exists match: resources: { kinds: [Pod] } verifyImages: - imageReferences:
['harbor.internal.corp/*'] attestations: - type: https://cyclonedx.org/bom attestors: -
```

```
entries: - keyless: subject: https://github.com/myorg/* issuer:  
https://token.actions.githubusercontent.com
```

CHAPTER 11

SLSA Framework for Container Builds

SLSA Overview

Supply-chain Levels for Software Artifacts (SLSA, pronounced 'salsa') defines a progressive security maturity model for software build and distribution. Each level adds requirements that protect against increasingly sophisticated attacks on the build process itself.

Level	Key Requirements	Protects Against	Implementation
SLSA 0	None	Nothing	Ad-hoc local builds
SLSA 1	Build process generates provenance	Accidental build tampering	GitHub Actions + provenance generator
SLSA 2	Hosted build, signed provenance by build service	Unauthorized build changes	GitHub Actions SLSA builder action
SLSA 3	Hardened builds: non-falsifiable provenance, isolated environment	Compromised CI runner	Ephemeral isolated builders, reproducible builds

SLSA Provenance -- What It Contains

```
SLSA provenance is a signed attestation describing how an artifact was built: provenance.json
(CycloneDX / in-toto predicate format): { buildType: https://slsa.dev/provenance/v1, builder:
{ id: https://github.com/actions/runner/releases/tag/v2.304.0 }, buildDefinition: {
externalParameters: { workflow: .github/workflows/build.yml, ref: refs/tags/v1.2.3,
repository: https://github.com/myorg/myapp } }, runDetails: { metadata: { invocationId:
https://github.com/myorg/myapp/actions/runs/12345, startedOn: 2025-06-01T12:00:00Z,
finishedOn: 2025-06-01T12:04:23Z } } } # Verify SLSA provenance: slsa-verifier verify-image \
harbor.internal.corp/myapp@sha256:DIGEST \ --source-uri github.com/myorg/myapp \ --source-tag
v1.2.3 \ --slsa-verifier-version v2.4.1
```

Tekton Chains -- SLSA Provenance in Kubernetes

Tekton Chains is a Kubernetes-native solution that automatically generates SLSA provenance for every Tekton Pipeline build, signs it with Cosign, and stores it as an OCI attestation -- enabling SLSA L2-L3 on-premises without dependency on GitHub Actions:

```
# Tekton Chains configuration for SLSA provenance: apiVersion: v1 kind: ConfigMap metadata:
name: chains-config namespace: tekton-chains data: artifacts.oci.format: slsa/v1
artifacts.oci.storage: oci signers.x509.fulcio.address: https://fulcio.internal.corp
transparency.url: https://rekor.internal.corp # Automatically signs every TaskRun output image
artifacts.pipelinerun.format: slsa/v1 artifacts.pipelinerun.storage: oci,gcs
```

CHAPTER 12

Vulnerability Scanning in Depth

Scanning Architecture for Kubernetes

Vulnerability scanning must occur at multiple stages of the container lifecycle, not just at build time. CVEs are discovered continuously; an image clean at build time may be critical within weeks.

Stage	When	Tools	Action on Critical CVE
Build time	Every CI build	Trivy, Gype, Snyk	Fail build; block registry push
Registry admission	On push to registry	Harbor + Trivy, ECR Inspector, GAR scanning	Block tag creation; alert security team
Pre-deployment	On every K8s admission	Kyverno image rules, OPA	Reject Pod; prevent deployment
Continuous (runtime)	Daily/weekly rescan of running images	Harbor scheduled scan, Anchore, Prisma	Alert; schedule forced rollout
SBOM-based rescan	On new CVE database update	Gype against stored SBOMs	Alert; correlate to running workloads

Trivy -- Production Scanning Configuration

```
# Scan image with full reporting: trivy image \ --severity CRITICAL,HIGH \ --exit-code 1 \
--format sarif \ --output trivy-results.sarif \ harbor.internal.corp/myapp:v1.2.3 # Scan with
ignorefile (accepted risk): trivy image \ --ignorefile .trivyignore \ --severity CRITICAL \
--exit-code 1 \ harbor.internal.corp/myapp:v1.2.3 # .trivyignore format: # CVE-2024-12345 #
accepted: no fix available, mitigated by network policy # CVE-2024-67890 # accepted: component
not reachable, expires 2025-01-01 # Scan filesystem (for CI before image build): trivy fs
--security-checks vuln,secret,config . # Scan Kubernetes cluster for vulnerable images: trivy
k8s --report summary cluster
```

Vulnerability Management Policy

Severity	SLA to Patch	CI Policy	Production Policy
Critical (CVSS 9.0+)	24-72 hours	Block build immediately	Alert + forced rollout within 24h
High (CVSS 7.0-8.9)	7-14 days	Block build (configurable)	Alert + scheduled rollout
Medium (CVSS 4.0-6.9)	30-90 days	Warn; track in ticket	Monitor; next release cycle

Severity	SLA to Patch	CI Policy	Production Policy
Low (CVSS 0.1-3.9)	Next major release	Log only	Track in backlog
No fix available	Accept + document	Allow with justification in .trivyignore	Mitigate via network policy

CHAPTER 13

Runtime Container Security

Defence in Depth for Running Containers

Runtime security addresses threats that persist after an image has been scanned and deployed: zero-day exploits, configuration drift, insider threats, and supply chain attacks not yet known to vulnerability databases. Multiple independent security layers provide defence in depth.

Falco -- Runtime Threat Detection

Falco (CNCF graduated) detects anomalous behaviour in running containers by monitoring system calls via eBPF probes. Unlike image scanning (static), Falco observes actual runtime behaviour.

```
# Example Falco rules for Kubernetes workloads: # Rule 1: Shell spawned in container (highly
suspicious) - rule: Shell spawned in container desc: A shell was spawned in a container
(possible compromise) condition: > spawned_process and container and proc.name in (bash, sh,
zsh, fish) and not proc.pname in (containerd-shim, runc) output: Shell spawned
(user=%user.name cmd=%proc.cmdline container=%container.id) priority: WARNING tags:
[container, shell, mitre_execution] # Rule 2: Unexpected network connection from
known-internal service - rule: Outbound connection from database container desc: Database
containers should not initiate outbound connections condition: > outbound and container and
k8s.pod.label.app in (postgres, mysql, redis) and fd.sport != 5432 and fd.sport != 3306
output: DB container unexpected outbound (dest=%fd.rip:%fd.rport) priority: CRITICAL # Rule 3:
Sensitive file access - rule: Read sensitive file in container condition: open_read and
container and fd.name in (/etc/shadow, /etc/passwd) output: Sensitive file read (file=%fd.name
container=%container.id) priority: WARNING
```

Falco Deployment in Kubernetes

```
# Install Falco with eBPF probe (preferred; no kernel module required): helm repo add
falcosecurity https://falcosecurity.github.io/charts helm install falco falcosecurity/falco \
--namespace falco --create-namespace \ --set driver.kind=ebpf \ --set
falcosidekick.enabled=true \ --set
falcosidekick.config.slack.webhookurl=https://hooks.slack.com/... \ --set
falcosidekick.config.pagerduty.routingKey=xxxx
```

Tetragon -- eBPF Security Enforcement

Tetragon (Cilium project) goes beyond Falco's detection-only model by enforcing security policy in the kernel via eBPF -- blocking malicious actions before they complete, not just alerting after the fact.

```
Tetragon TracingPolicy to block /bin/bash execution in production pods: apiVersion:
cilium.io/v1alpha1 kind: TracingPolicy metadata: name: block-shell-execution spec: kprobes: -
call: sys_execve syscall: true args: - index: 0 type: string selectors: - matchArgs: - index:
0 operator: Equal values: ["/bin/sh", "/bin/bash", "/usr/bin/bash"] matchNamespaces: -
namespace: Mnt operator: NotIn values: ["host_mnt_ns_id"] matchActions: - action: Sigkill #
Kill the process attempting to exec shell
```

Security Context Best Practices

```
Complete hardened Pod security context (Pod Security Standards: Restricted): apiVersion: v1
kind: Pod metadata: name: hardened-app annotations:
container.apparmor.security.beta.kubernetes.io/app: runtime/default spec: securityContext:
runAsNonRoot: true runAsUser: 1000 runAsGroup: 1000 fsGroup: 1000 seccompProfile: type:
RuntimeDefault containers: - name: app image: harbor.internal.corp/myapp@sha256:DIGEST
securityContext: allowPrivilegeEscalation: false readOnlyRootFilesystem: true capabilities:
drop: [ALL] seccompProfile: type: RuntimeDefault volumeMounts: - name: tmp mountPath: /tmp #
writable tmpfs for temp files - name: cache mountPath: /app/cache # writable cache directory
volumes: - name: tmp emptyDir: { medium: Memory } - name: cache emptyDir: {}
```

CHAPTER 14

Container Anti-Patterns and Remediation

Anti-Pattern: Running containers as root

Problem: Container process runs as UID 0. If container escapes (via kernel exploit or misconfiguration), attacker has root on the node.

Solution: Set `runAsNonRoot: true` and `runAsUser: 1000+` in `securityContext`. Build images with `USER` instruction. Enforce via Pod Security Standards Restricted.

Anti-Pattern: Storing secrets in environment variables

Problem: Environment variables are visible in `docker inspect`, `kubectl describe pod`, and any process that can read `/proc/self/environ`. Secrets in env vars are also logged by many frameworks.

Solution: Use Kubernetes Secrets mounted as files (not env vars where possible). Use External Secrets Operator syncing from Vault. Mount secrets to tmpfs paths (`/run/secrets/`).

Anti-Pattern: Mutable container images (tag-based deployments)

Problem: Deploying with `:latest` or `:v1.2` tags. Tags are mutable pointers; the underlying image can change without your awareness, enabling supply chain substitution attacks.

Solution: Always pin images by SHA-256 digest in production PodSpecs. Use Kyverno `mutateDigest` to automatically resolve tags to digests at admission.

Anti-Pattern: Privileged containers

Problem: `spec.containers[].securityContext.privileged: true` gives the container full host access -- effectively root on the node. Used (incorrectly) when developers need capabilities they cannot get another way.

Solution: Identify the specific capability needed; add only that capability. For DaemonSet node agents, use `hostPID/hostNetwork` minimally. Enforce no-privileged via Pod Security Standards.

Anti-Pattern: Mounting Docker socket

Problem: Mounting `/var/run/docker.sock` into a container gives it full control over the Docker daemon -- and therefore all containers on the node. Used in CI/CD Docker-in-Docker patterns.

Solution: Use Kaniko, BuildKit (rootless), or Tekton for in-cluster builds. Use external CI (GitHub Actions) for image builds. Never mount Docker socket in production workloads.

Anti-Pattern: Single large container instead of microservices

Problem: Running a large application as a single container misses K8s benefits: independent scaling, independent deployment, independent failure domains.

Solution: Decompose over time using Strangler Fig pattern. Start with multi-container Pods (sidecar pattern) before full decomposition. Not every app benefits from decomposition -- assess ROI.

Anti-Pattern: Ignoring ephemeral storage limits

Problem: Applications write large files to container filesystem or emptyDir without limits, causing node disk exhaustion -- a node-level failure affecting all Pods on the node.

Solution: Set `resources.limits.ephemeral-storage`. Use PVCs for large/persistent writes. Mount emptyDir with `sizeLimit` for temporary storage. Monitor node disk usage with Prometheus `node_filesystem_avail_bytes`.

CHAPTER 15

Hands-On Exercises

Exercise 3.1 -- Multi-Stage Build Optimisation

Compare image sizes between naive and optimised build strategies:

```
# 1. Build a naive single-stage Python image: cat > Dockerfile.naive << 'EOF' FROM python:3.12
WORKDIR /app COPY . . RUN pip install -r requirements.txt CMD ["python", "app.py"] EOF docker
build -t myapp:naive -f Dockerfile.naive . docker images myapp:naive # typically 1.0-1.5 GB #
2. Build an optimised distroless multi-stage image: cat > Dockerfile.optimised << 'EOF' FROM
python:3.12-slim AS builder WORKDIR /build COPY requirements.txt . RUN pip install
--prefix=/install -r requirements.txt FROM gcr.io/distroless/python3-debian12:nonroot COPY
--from=builder /install /usr/local COPY app.py /app/app.py WORKDIR /app USER nonroot:nonroot
CMD ["app.py"] EOF docker build -t myapp:optimised -f Dockerfile.optimised . docker images
myapp:optimised # typically 60-100 MB # 3. Inspect layers with dive: docker run --rm -it -v
/var/run/docker.sock:/var/run/docker.sock \ wagoodman/dive:latest myapp:optimised
```

Exercise 3.2 -- Image Signing with Cosign

Sign a container image and enforce signature verification in Kubernetes:

```
# 1. Install cosign: brew install cosign # macOS # or: go install
github.com/sigstore/cosign/v2/cmd/cosign@latest # 2. Sign with a local key (for lab; use
keyless in production): cosign generate-key-pair # Creates cosign.key and cosign.pub # 3. Push
and sign an image: docker tag myapp:optimised ttl.sh/myapp:1h # ttl.sh = ephemeral public
registry docker push ttl.sh/myapp:1h IMAGE_DIGEST=$(docker inspect --format='{{index
.RepoDigests 0}}' ttl.sh/myapp:1h) cosign sign --key cosign.key $IMAGE_DIGEST # 4. Verify the
signature: cosign verify --key cosign.pub $IMAGE_DIGEST # 5. (Advanced) Scan signature in
Kubernetes with Kyverno: # Install Kyverno, apply ClusterPolicy from Chapter 9, # then try
deploying a signed vs unsigned image and observe enforcement
```

Exercise 3.3 -- Supply Chain Security Audit

Perform a supply chain security audit of a production image:

```
# 1. Scan for CVEs: trivy image nginx:alpine # 2. Generate SBOM: syft nginx:alpine -o
cyclonedx-json=nginx-sbom.cdx.json cat nginx-sbom.cdx.json | jq '.components | length' # Count
how many packages are in the image # 3. Check for secrets in image: trivy image
--security-checks secret nginx:alpine # 4. Inspect image configuration for security issues:
trivy image --security-checks config nginx:alpine # 5. Verify OCI manifest structure: crane
manifest nginx:alpine | jq . crane ls nginx # list available tags # 6. Check if image is
signed: cosign verify --certificate-identity-regex . \ --certificate-oidc-issuer-regex . \
nginx:alpine 2>&1 | head -5 # nginx:alpine is NOT signed -- demonstrates why internal mirror
matters
```

End of Part III -- [Continue to Part IV: Kubernetes Internals](#)

Part IV delivers the definitive deep-dive into Kubernetes architecture internals: API Server, etcd, Scheduler, Controller Manager, Cloud Controller Manager, kubelet, kube-proxy, CoreDNS, Admission Controllers, CRDs, Operators, reconciliation loops, scheduling lifecycle, leader election, and HA design. This is the architectural foundation required to make informed production decisions and diagnose complex cluster behaviour.